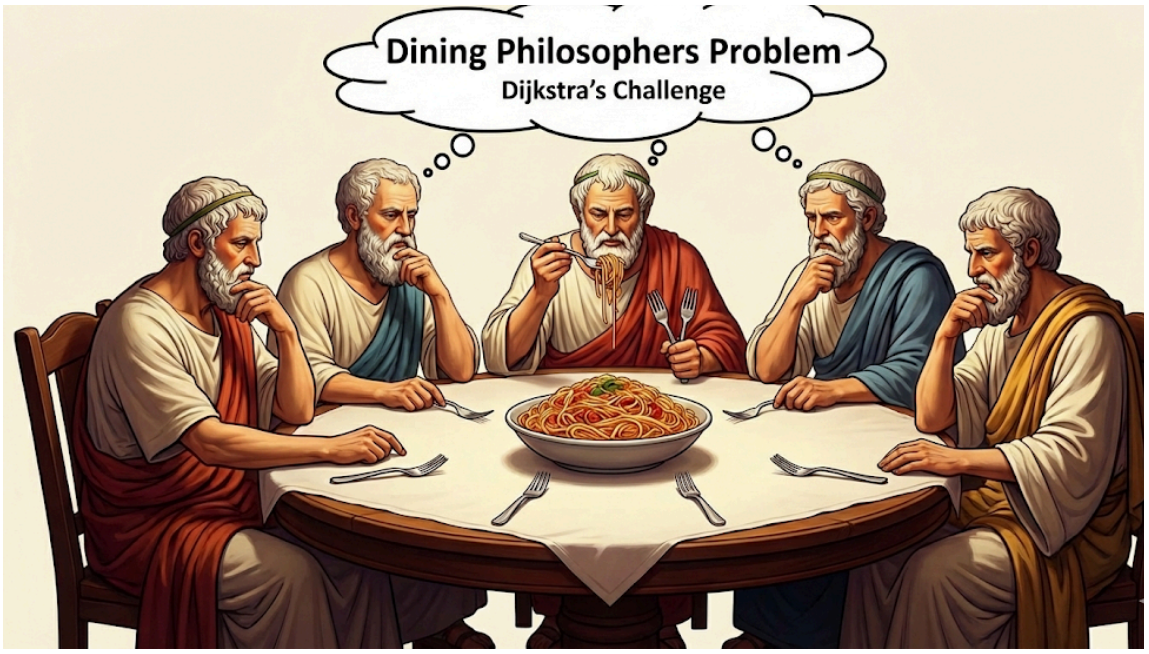
Lecture 15 NOTES – Classic Synchronization Problems II: Dining Philosophers and Monitors

The Dining Philosophers problem

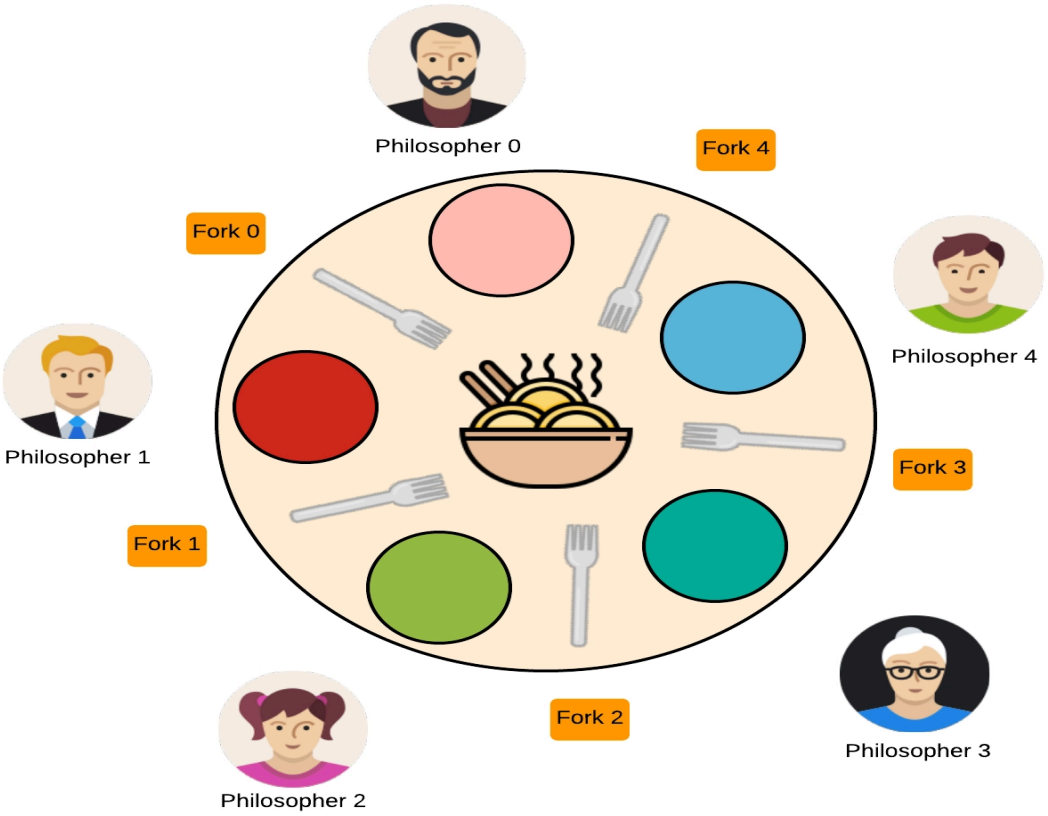
- **What is it?** The Dining Philosophers problem is a classic theoretical scenario used in computer science to test synchronization algorithms. It was formulated by Edsger Dijkstra to model the challenges of allocating limited resources among competing processes.



- **The Scenario:**
    1. Imagine **5 philosophers** sitting around a round dining table.
    2. There is a bowl of noodles (or spaghetti) in the center.
    3. There are **5 chopsticks** (or forks) placed between them.
    4. **The Rule:** To eat, a philosopher needs **two chopsticks**—one from their left and one from their right. A philosopher can only do two things: **Think** or **Eat**.
- **Resource Contention Problem:** The philosophers represent **processes** (programs), and the chopsticks represent **shared resources** (like files, I/O devices, or memory locks).
    1. **Problem:** If everyone gets hungry at the same time, there aren't enough resources (chopsticks) for everyone to eat simultaneously.
- **What it Demonstrates:** This problem is famous because a bad solution leads to two major system failures:

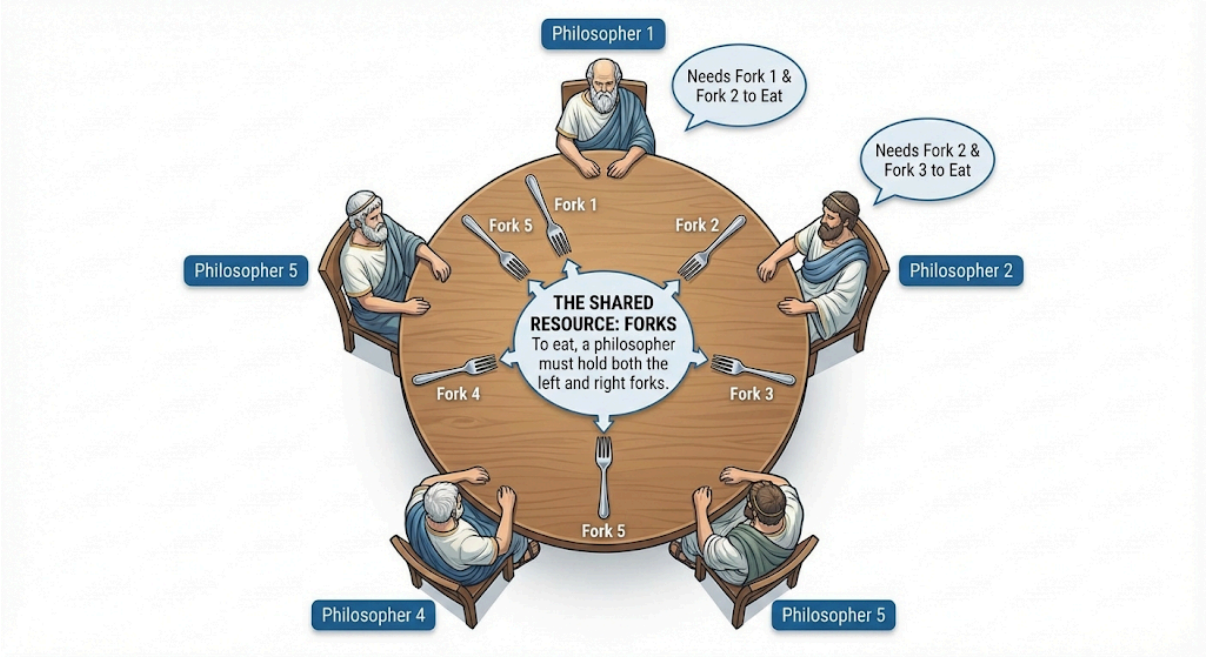
1. **Deadlock:** Every philosopher picks up their left chopstick at the same time. Now, everyone waits for the right chopstick, which is held by their neighbor. No one can eat, and they wait forever.
2. **Starvation:** A philosopher might get stuck waiting indefinitely while others keep eating, never getting a turn.

- **Motivation for Monitors:** Solving this problem using simple tools like Semaphores is tricky and prone to errors (like deadlock). This difficulty motivated the invention of **Monitors**—a higher-level synchronization tool that handles the complex locking logic automatically, making it much harder for programmers to make mistakes.



Problem Statement

- **The Setup:**
  1. **5 Philosophers** sit at a round table.
  2. **5 Forks** (often called chopsticks) are placed between them. Each philosopher has one fork to their immediate left and one to their immediate right.
  3. **The Shared Resource:** The forks. To eat, a philosopher *must* hold **both** the left and right forks.



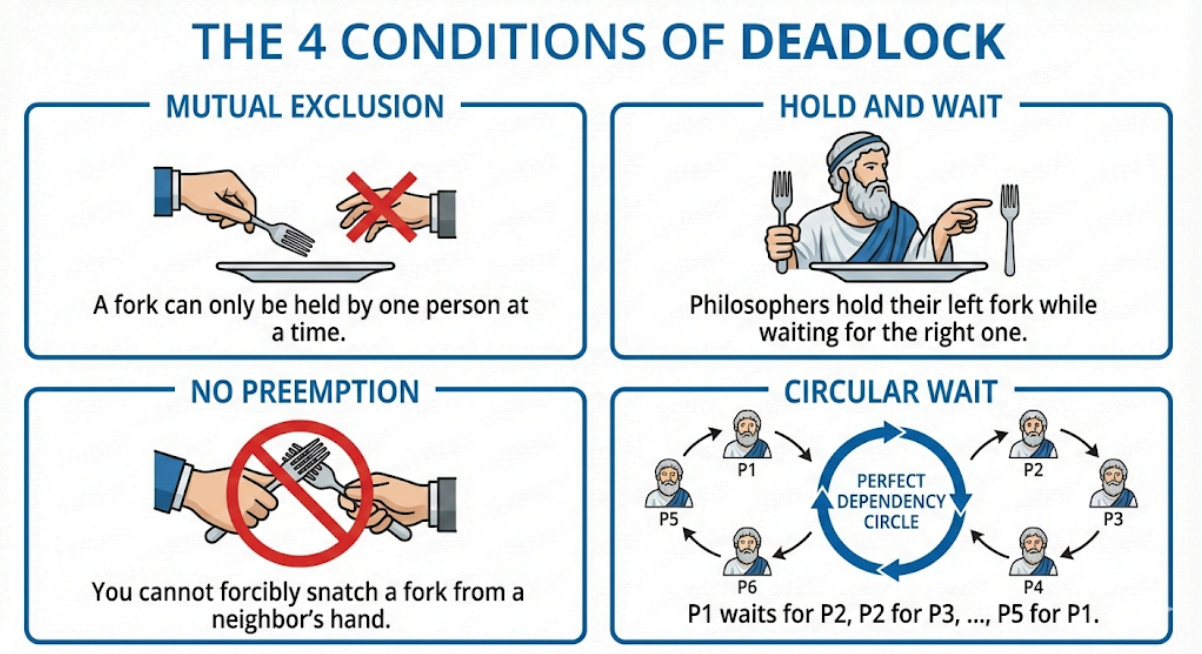
- **Philosopher States:**
    1. **Thinking:** Doing nothing, needing no resources.
    2. **Hungry:** Wants to eat. Tries to pick up forks.
    3. **Eating:** Holding both forks, consuming food. (After eating, they put forks down and go back to thinking).

Naive Solution (The Trap)

- **The Logic:** Every philosopher follows this simple rule:
    - Pick up **Left** fork.
    - Pick up **Right** fork.
    - Eat.
    - Put down forks.
- **The Issue:** This leads to **Deadlock**.
    - Imagine all 5 philosophers become hungry at the exact same moment.
    - Everyone picks up their **Left** fork simultaneously.
    - Now, everyone reaches for their **Right** fork.
    - **Result:** Every right fork is already held by the neighbor. Everyone waits forever.

Deadlock Explanation (The 4 Conditions)

This problem is the textbook example of Deadlock because it satisfies all four necessary conditions simultaneously:



1. **Mutual Exclusion:** A fork can only be held by one person at a time.
2. **Hold and Wait:** Philosophers hold their left fork while waiting for the right one.
3. **No Preemption:** You cannot forcibly snatch a fork from a neighbor's hand.
4. **Circular Wait:** P1 waits for P2, P2 waits for P3, ..., P5 waits for P1. A perfect circle of dependency.

Semaphore-Based Solutions

Solution 1: The Asymmetric Solution

1. The Core Concept The root cause of deadlock in the Dining Philosophers problem is the Circular Wait condition. If every philosopher sits at the table and performs the exact same action simultaneously (e.g., "everyone pick up the left fork"), the entire table creates a closed circle of dependency.

To solve this, we introduce Asymmetry. We make one philosopher behave differently from the rest. By changing the rules for just one person, we prevent the circle from closing.

2. The Strategy (The "Right-Handed" Rebel)

- **Philosophers 0 to 3:** These are "Left-Handed" thinkers. They pick up their Left chopstick first, then their Right.
- **Philosopher 4 (The Last One):** This is the "Right-Handed" rebel. They pick up their Right chopstick first, then their Left.

3. Why Deadlock is Impossible (The Trace) Imagine a worst-case scenario where everyone gets hungry at the exact same moment:

1. **P0, P1, P2, and P3** all successfully grab their Left chopsticks (C0, C1, C2, C3).

2. **P4** tries to behave differently. They want to grab their Right chopstick (C0) first.
3. **The Conflict:** C0 is already held by P0.
4. **The Result:** P4 blocks (sleeps) waiting for C0. Crucially, P4 has NOT picked up C4 yet.
5. **The Breakthrough:** Because P4 is stuck waiting for the first step, chopstick C4 remains free on the table.
6. **P3's Opportunity:** P3 is holding C3 and looking for C4. Since C4 is free, P3 grabs it, eats, and eventually puts both down.
7. **The Chain Reaction:** Once P3 finishes, P2 can eat, then P1, then P0. Finally, P0 drops C0, waking up P4.

👤 **Philosopher Code (Asymmetric)**

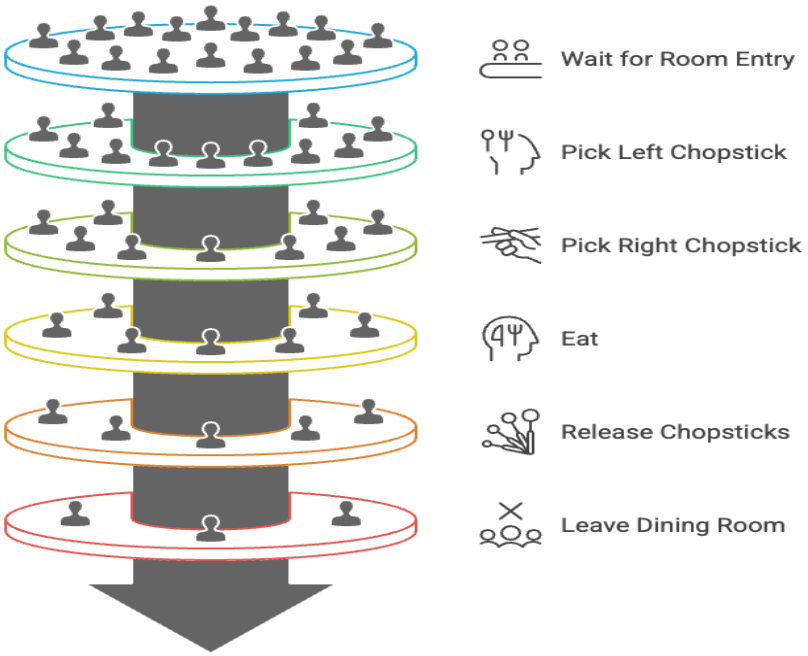
```
void philosopher(int i) {
   while (true) {
       think();
       if (i == 4) {
           // Last philosopher picks RIGHT first
           wait(chopstick[(i+1) % 5]);
           wait(chopstick[i]);
       } else {
           // Others pick LEFT first
           wait(chopstick[i]);
           wait(chopstick[(i+1) % 5]);
       }
       eat();
       signal(chopstick[i]);
       signal(chopstick[(i+1) % 5]);
   }
}
```

✅ **Why This Works**

- Philosopher **4** picks chopsticks in **opposite order**
- **Circular wait condition is broken**
- Not all philosophers can hold one chopstick simultaneously
- ✔ **Deadlock is prevented**

Solution 2: The "N-1" Capacity Solution

1. The Core Concept (The Pigeonhole Principle) Deadlock happens in the Dining Philosophers problem when *all 5* philosophers pick up their left chopstick at the same time. There are 5 people and 5 chopsticks, so everyone holds 1 and waits for 1. The system freezes.



To fix this, we change the rules: **We only allow 4 philosophers to sit at the table at once.**

- **The Logic:** If there are 5 chopsticks and only 4 people, it is mathematically impossible for everyone to be holding 1 chopstick and waiting.
- **The Guarantee:** Even in the worst-case scenario (all 4 seated people grab their left chopstick), there is still **1 chopstick remaining** on the table. One of the 4 philosophers *must* be able to grab it, eat, and finish.

2. The Implementation (The "Room" Semaphore) We introduce a new Counting Semaphore called `room` (or `footman`).

- **Initial Value:** We initialize `room = 4` (Maximum capacity).
- **Entry Rule:** Before a philosopher can even try to pick up a chopstick, they must first acquire a "seat" by calling `wait(room)`.
- **Exit Rule:** When they finish eating, they release their seat with `signal(room)`.

```
semaphore chopstick[5] = {1, 1, 1, 1, 1};
semaphore room = 4;   // Only 4 philosophers allowed

👤 Philosopher Code
void philosopher(int i) {
   while (true) {
     think();
     wait(room);            // Enter dining room
     wait(chopstick[i]);    // Pick left chopstick
     wait(chopstick[(i+1) % 5]);// Pick right chopstick
     eat();
     signal(chopstick[i]);
     signal(chopstick[(i+1) % 5]);
     signal(room);                // Leave dining room
```

```
    }
}
```

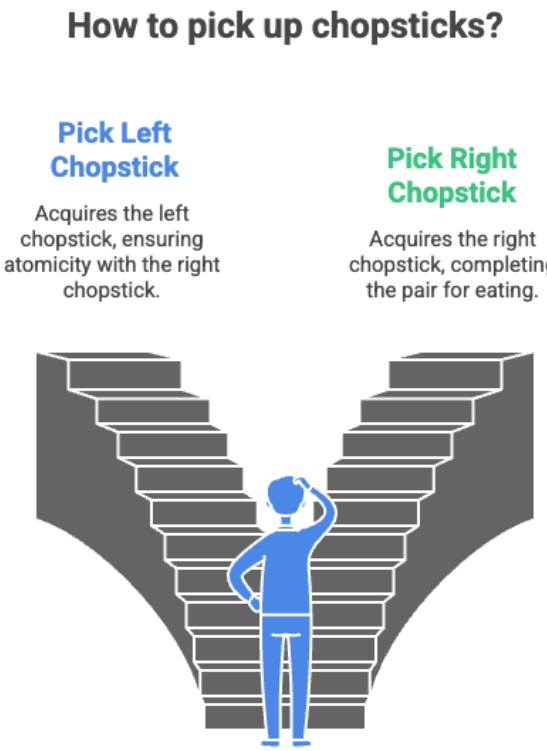
✅ **Why This Works**

- At most **N−1 philosophers** compete for chopsticks
- At least **one philosopher gets both chopsticks**
- Circular wait is **impossible**
- Ensures **progress** (no deadlock)

Solution 3: The Global Mutex Solution (All or Nothing)

1. The Core Concept (Atomicity) The danger in the Dining Philosophers problem is "Partial Allocation"—a philosopher picking up one chopstick and then getting stuck waiting for the second one. This holding-while-waiting creates the deadlock.

To fix this, we enforce an **"All or Nothing"** rule. A philosopher is only allowed to pick up chopsticks if they can pick up **both** of them at the exact same time. We achieve this by using a global `mutex` lock that protects the act of grabbing resources.



2. The Mechanism

- **The Global Lock:** We add a binary semaphore called `mutex`.
- **The Critical Section:** The entire sequence of picking up the Left and Right chopsticks is placed inside this lock.

- **The Result:** When Philosopher 0 is picking up forks, no one else is allowed to even *try* to pick up forks. They must wait for the mutex. This guarantees that if you start picking up forks, you will succeed in getting both without being interrupted or blocked by a neighbor holding just one.

3. Code

```
semaphore chopstick[5] = {1, 1, 1, 1, 1};
semaphore mutex = 1;   // Protects critical section

👤 Philosopher Code
void philosopher(int i) {
   while (true) {
     think();
     wait(mutex);               // Enter critical section
     wait(chopstick[i]);        // Pick left chopstick
     wait(chopstick[(i+1) % 5]); // Pick right chopstick
     signal(mutex);             // Exit critical section
     eat();
     wait(mutex);               // Enter critical section
     signal(chopstick[i]);      // Put left chopstick
     signal(chopstick[(i+1) % 5]); // Put right chopstick
     signal(mutex);             // Exit critical section
   }
}
```

4. Why This Works

- **Prevents Hold-and-Wait:** You cannot hold one chopstick and wait for another *while preventing others from checking*. Because the acquisition is atomic (protected by the mutex), you effectively grab both simultaneously from the perspective of the other threads.
- **No Deadlock:** It is impossible to get into a state where everyone is holding one chopstick, because the `mutex` allows only one person to transition from "Thinking" to "Eating" logic at a time.

5. Pros and Cons

- **Pro (Safety):** Deadlock is mathematically impossible.
- **Pro (Simplicity):** Very easy to code.
- **Con (Performance Bottleneck):** This solution limits concurrency during the *pickup* phase. Even if Philosopher 0 and Philosopher 2 (who sit far apart) want to eat, P2 must wait for P0 to finish picking up forks before P2 can even try, even though they don't share chopsticks. This serialization can reduce performance in very large systems.

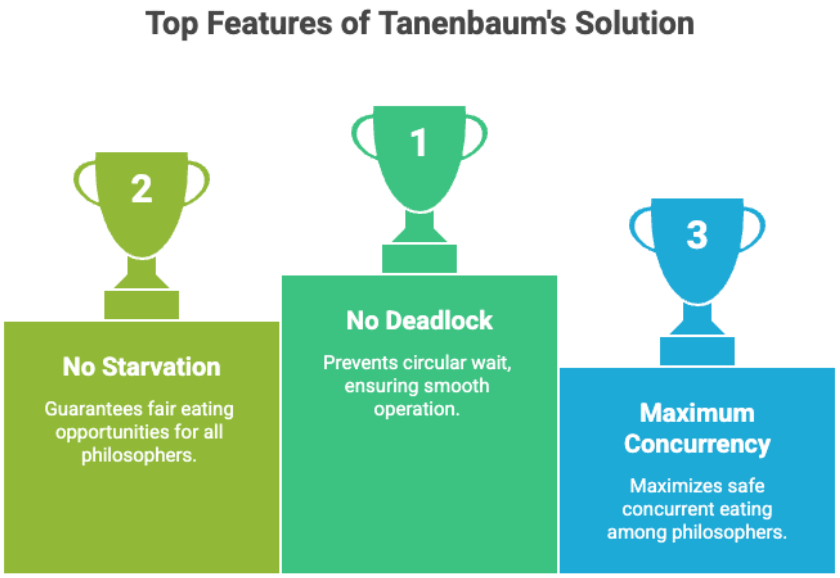
6. Summary Checklist

- **Goal:** Eliminate partial resource allocation.

- **Method:** Wrap the `wait()` calls in a global `mutex`.
- **Key Detail:** Keep `eat()` **outside** the mutex to allow parallel eating.
- **Outcome:** Safe from deadlock, but slightly less efficient than the Asymmetric Solution.

Here is the detailed content for **Solution 4: Check Both Chopsticks (Tanenbaum's Solution)**. This is often considered the most sophisticated solution because it allows for maximum concurrency without deadlock.

Solution 4: State-Based Checking (Tanenbaum's Solution)



1. The Core Concept (Smart Checking) Instead of blindly trying to pick up chopsticks one by one, this solution changes the perspective. It focuses on the **State of the Philosopher** rather than the locks on the chopsticks.

- **The Logic:** A philosopher is allowed to move to the **EATING** state *only if* neither of their neighbors is currently **EATING**.
- **The "Test":** Before I eat, I check my Left Neighbor and my Right Neighbor.
  - If they are busy eating, I wait.
  - If they are not eating, I take both chopsticks instantly.

2. The Helper Functions This solution divides the logic into three specific functions:

1. `take_forks(i)`: I want to eat. I declare myself HUNGRY. I try to get forks. If I fail, I sleep.
2. `put_forks(i)`: I am done. I declare myself THINKING. Then, I behave nicely and **check if my neighbors want to eat** (waking them up if they do).
3. `test(i)`: The critical check. It looks at `state[i]`, `state[LEFT]`, and `state[RIGHT]`.

3.Code

🔄 **Philosopher States**

```
#define THINKING 0
#define HUNGRY   1
#define EATING   2
```

🧰 **Shared Data Structures**

```
int state[5];                  // State of each philosopher
semaphore mutex = 1;           // Protects critical section
semaphore s[5] = {0,0,0,0,0}; // One semaphore per philosopher
```

📍 **Helper Macros**

```
#define LEFT  (i+4) % 5
#define RIGHT (i+1) % 5
```

4. Why This Works

- **Deadlock Free:** You can never get stuck holding one chopstick while waiting for another. The state transition to `EATING` is atomic (protected by mutex). You either get both forks or you get none.
- **Maximum Concurrency:** It allows the maximum possible number of philosophers (2) to eat simultaneously, regardless of position (e.g., P0 and P2 can eat, or P1 and P3).
- **Communication:** It uses the `s[i]` semaphore not for mutual exclusion, but for **Signaling**. `put_forks` actively looks to wake up blocked neighbors, ensuring the system keeps moving.

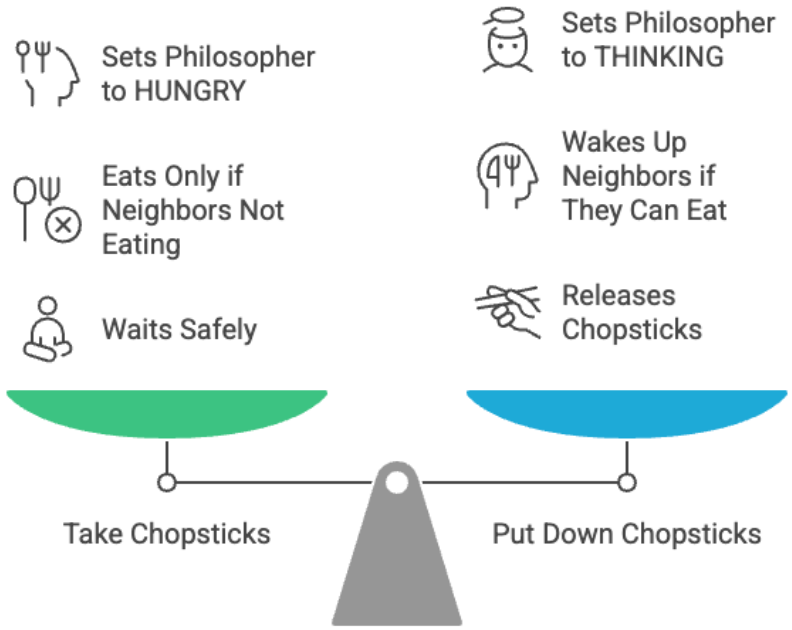
5. Pros and Cons

- **Pro:** Deadlock-free and allows max parallelism.
- **Con:** It is possible (though rare) for **Starvation** to occur. If P1 and P3 alternate eating perfectly such that P2 always sees one of them eating whenever P2 checks, P2 might never get a turn. (This requires perfect, unlucky timing).

6. Summary Checklist

- **Goal:** Efficient checking of neighbors.
- **Method:** Track `state[]` and use `test()` to check neighbors.
- **Key Detail:** `put_forks` checks neighbors, not just self.
- **Outcome:** Deadlock impossible; starvation theoretically possible but unlikely.

Tanenbaum's Solution:



1. `take_chopsticks(i)` – The Request Protocol

This function is called when a philosopher gets hungry. Its goal is to transition the philosopher from **THINKING** to **EATING**, blocking them if the resources aren't available.

🍴 **Take Chopsticks**

```
void take_chopsticks(int i) {
   wait(mutex);
   state[i] = HUNGRY;
   test(i);      // Try to acquire both chopsticks
   signal(mutex);
   wait(s[i]);   // Block if not allowed to eat
}
```

Key Insight: The "Sleep" Mechanism The most clever part of this code is the separation of the `mutex` and the private semaphore `s[i]`.

- We hold the `mutex` **only** long enough to check the status.
- If we can't eat, we release the `mutex` **before** we go to sleep at `wait(s[i])`. This ensures we don't hold the lock while sleeping (which would cause a deadlock).

2. `put_chopsticks(i)` – The Release Protocol

This function is called when a philosopher finishes eating. Its goal is to release resources and **proactively wake up** any neighbors who might have been waiting for these chopsticks.

🍽️ **Put Down Chopsticks**

```
void put_chopsticks(int i) {
   wait(mutex);
   state[i] = THINKING;
   test(LEFT);      // Check left neighbor
   test(RIGHT);     // Check right neighbor
   signal(mutex);
}
```

Key Insight: Active Notification In the simple solutions, a philosopher just puts down a fork and walks away. In Tanenbaum's solution, the philosopher plays an active role in scheduling.

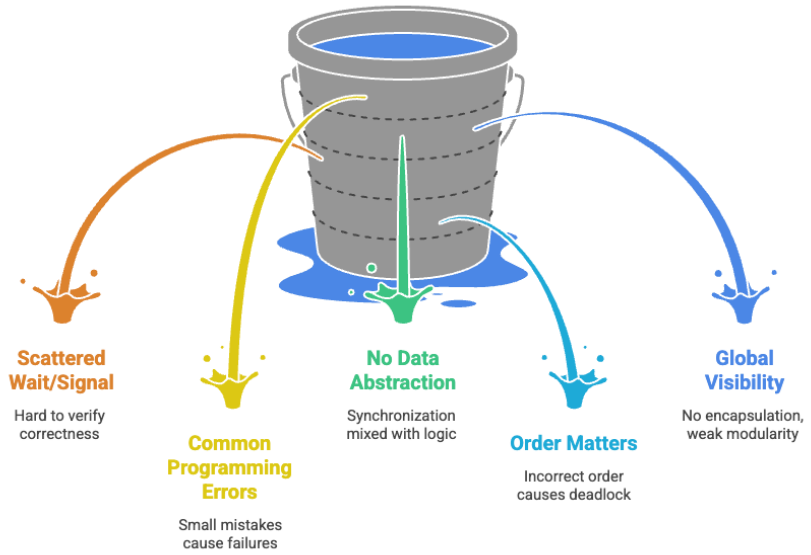
- By calling `test(LEFT)`, **I am acting as the scheduler** for my neighbor.
- If `test(LEFT)` succeeds, it executes `signal(s[LEFT])`. This wakes up my neighbor who is currently sleeping in their own `take_chopsticks()` function.

Summary of the Workflow

1. **P1 gets Hungry:** Calls `take_chopsticks`. Sets state to `HUNGRY`. Calls `test`. Neighbors are eating. `test` fails. P1 blocks at `wait(s[1])`.
2. **P2 Finishes:** Calls `put_chopsticks`. Sets state to `THINKING`. Calls `test(P1)`.
3. **The Handoff:** P2 (running `test(P1)`) sees that P1 is `HUNGRY` and neighbors are clear. P2 sets P1's state to `EATING` and calls `signal(s[1])`.
4. **P1 Wakes Up:** P1 unblocks from `wait(s[1])` and starts eating.

The Downside of Semaphores

While Semaphores are powerful and can solve any synchronization problem, they are considered a **low-level mechanism**. Using them is like writing code in Assembly language: it works, but it is dangerous, error-prone, and hard to read.



🔀 1. Scattered Synchronization (The "Spaghetti" Problem)

- **The Issue:** wait() and signal() operations are often scattered throughout the entire codebase. They are not grouped together in one place.

- **The Consequence:** If you have a bug (e.g., a race condition), you cannot just look at one "Synchronization Class." You have to search through thousands of lines of code to find every instance where a semaphore was touched.
- **Maintenance Nightmare:** It is extremely difficult to verify that the system is correct just by reading the code.

🧨 2. Common Programming Errors (The "Human Factor")

Because semaphores rely on the programmer to manually place wait and signal calls, simple typos lead to catastrophic system failures. The compiler cannot catch these logic errors.

- **Violation of Mutual Exclusion:**
  - *Code:* signal(mutex); ... critical_section ... wait(mutex);
  - *Result:* The programmer accidentally swapped the calls. The door is unlocked *before* entering. Multiple threads enter the critical section, causing data corruption.
- **Deadlock (Self-Blocking):**
  - *Code:* wait(mutex); ... wait(mutex);
  - *Result:* The thread acquires the lock, then tries to acquire it *again*. It waits forever for itself to release the lock.
- **The "Lost Key" (Omission):**
  - *Code:* wait(mutex); ... return;
  - *Result:* The programmer returns from a function but forgets to call signal(mutex). The resource remains locked forever, and the system eventually halts.

🧩 3. Lack of Data Abstraction

- **The Issue:** A semaphore is just an integer. It has no connection to the data it protects.
- **The Risk:** You can accidentally use the "Printer Mutex" to protect the "Database." The compiler won't stop you because it's just a number.
- **Logic Mixing:** Synchronization code (wait/signal) is mixed directly with business logic (calculating data), making the code hard to read.

🔄 4. Order Dependency (Fragility)

- **The Issue:** As seen in the Producer-Consumer problem, the *order* in which you call wait matters immensely.
- **The Risk:** wait(empty) followed by wait(mutex) is safe. Swapping them causes a deadlock.
- **Debugging:** These bugs are often "Heisenbugs"—they only appear under specific timing conditions, making them incredibly hard to reproduce and fix.

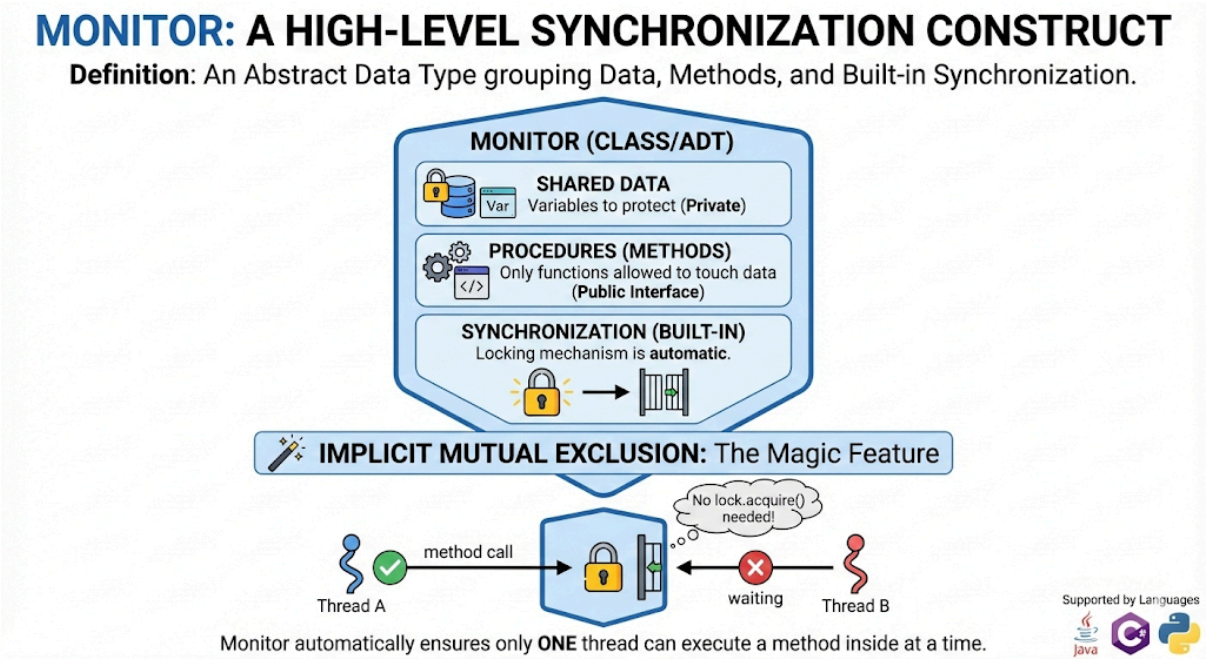
🌍 5. Global Visibility (No Encapsulation)

- **The Issue:** Semaphores are usually global variables. Any function in the program can call wait() or signal() on them, even functions that shouldn't have access to that resource.
- **The Consequence:** A malicious or poorly written plugin could call signal(mutex) randomly, unlocking resources that other threads are using, crashing the whole system.

Summary Table: Semaphores vs. High-Level Constructs

Feature	Semaphores (Low Level)	Monitors (High Level)
Control	Manual (wait/signal)	Automatic (Compiler handles it)
Location	Scattered everywhere	Centralized in one class
Safety	Easy to deadlock/error	Harder to make mistakes
Analogy	**GOTO statements**	**Functions/Methods**

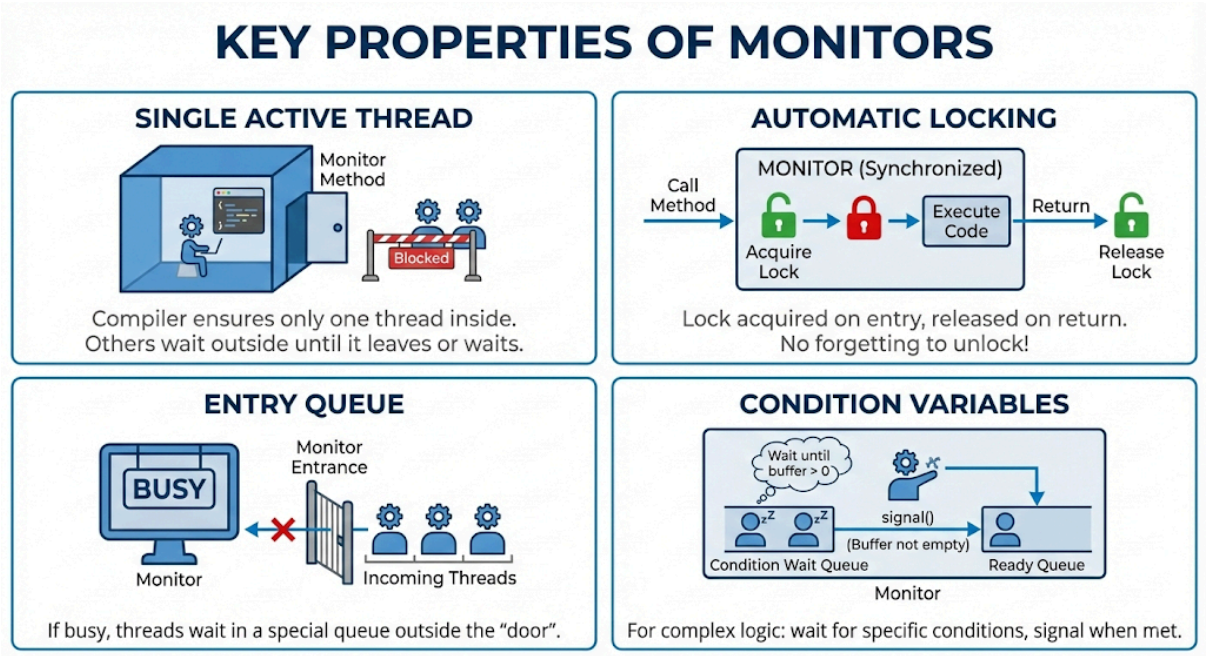
What is a Monitor?



- **Definition:** A Monitor is a **high-level synchronization construct** (supported by languages like Java, C#, and Python).

- **The Concept:** It is an abstract data type (like a Class) that groups three things together:
    1. **Shared Data:** The variables you want to protect.
    2. **Procedures (Methods):** The only functions allowed to touch that data.
    3. **Synchronization:** The locking mechanism is built-in.
- **Implicit Mutual Exclusion:** This is the magic feature. You don't write lock.acquire(). The Monitor automatically ensures that **only one thread** can execute a method inside the monitor at a time.

Key Properties of Monitors



1. **Single Active Thread:** The compiler ensures that if Thread A is inside a Monitor method, Thread B is blocked outside until Thread A leaves or waits.
2. **Automatic Locking:** The lock is acquired when you call a method and released when you return. No more forgetting to unlock!
3. **Entry Queue:** If the Monitor is busy, incoming threads wait in a special queue outside the "door."
4. **Condition Variables:** Used for more complex waiting logic (like "wait until the buffer is not empty").

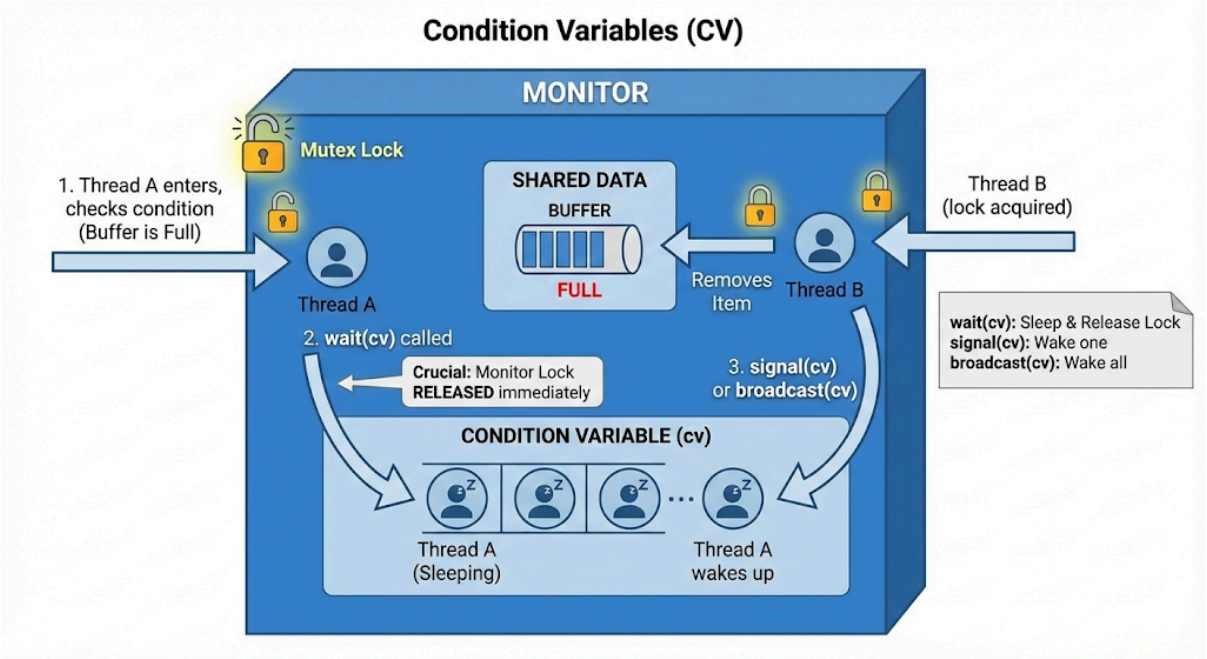
Condition Variables (CV)

Inside a Monitor, simple locking isn't enough. sometimes a thread enters, realizes it can't do its job yet (e.g., Buffer is Full), and needs to wait *inside* the monitor.

We use special variables (e.g., cv) that support three operations:

1. **wait(cv)**:

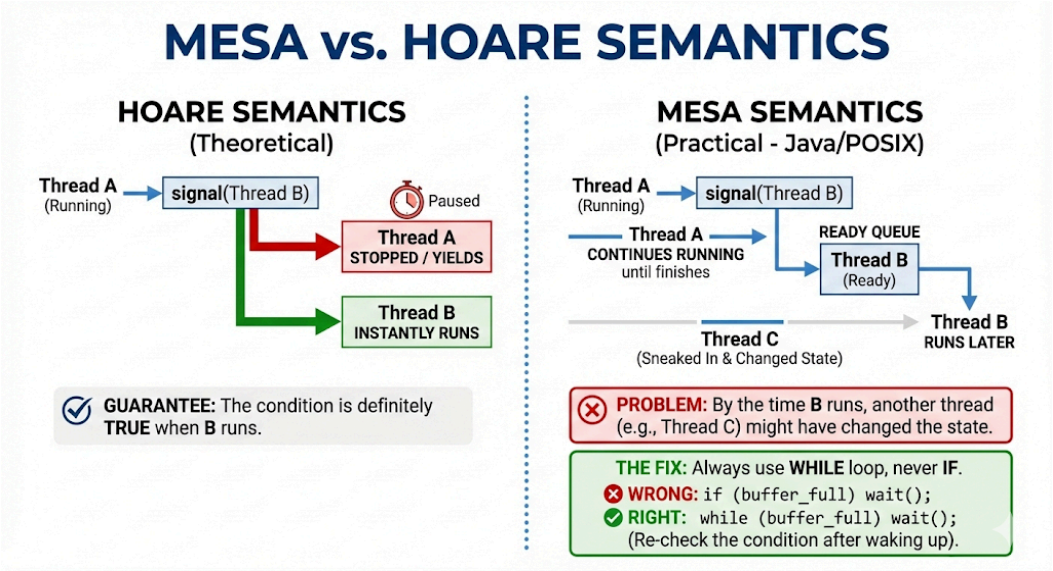
- The thread goes to sleep.
- **Crucial:** It **releases the monitor lock** immediately so someone else can enter.



2. **signal(cv)**:
    - Wakes up **one** thread that is sleeping on this variable.
    - If no one is waiting, the signal is lost (unlike semaphores, which count signals).
3. **broadcast(cv)** (or notifyAll):
    - Wakes up **all** threads waiting on this variable.

Mesa vs. Hoare Semantics

When Thread A signals Thread B, what happens next?



- **Hoare Semantics (Theoretical):**
    - Thread A stops immediately. Thread B runs instantly.
    - *Guarantee:* The condition is definitely true when B runs.

- **Mesa Semantics (Practical - Used in Java/POSIX):**
    - Thread A continues running until it finishes. Thread B is moved to the Ready Queue and runs later.
    - *Problem:* By the time B runs, another thread (Thread C) might have sneaked in and changed the state.
    - *The Fix:* **Always use while loop, never if.**
        - *Wrong:* if (buffer_full) wait();
        - *Right:* while (buffer_full) wait(); (Re-check the condition after waking up).

Monitor-Based Solution

This code is much cleaner than the Semaphore version.

The Setup:

- **Monitor Name:** BoundedBuffer
- **Condition Variables:**
    - not_full: "Wait here if the buffer is full."
    - not_empty: "Wait here if the buffer is empty."

Producer Code (insert method):

```
public synchronized void insert(Item item) {
    while (count == N) {
        not_full.wait(); // Buffer full? Go to sleep & release lock.
    }

    buffer[in] = item;   // Add item
    count++;

    not_empty.signal();  // Wake up a sleeping Consumer
}
```

Consumer Code (remove method):

Java

```
public synchronized void remove() {
    while (count == 0) {
        not_empty.wait(); // Buffer empty? Go to sleep & release lock.
    }

    item = buffer[out];   // Take item
    count--;

    not_full.signal();    // Wake up a sleeping Producer
}
```

Monitors vs. Semaphores (Comparison)

Aspect	Semaphores	Monitors
Level	**Low-Level** (Machine/OS).	**High-Level** (Programming Language).
Mutual Exclusion	**Explicit.** You must write wait(mutex) and signal(mutex).	**Implicit.** The language handles locking automatically.
Safety	**Error-Prone.** Easy to cause deadlocks or race conditions.	**Safe.** Much harder to make locking mistakes.
Readability	**Hard.** Logic is scattered.	**Easy.** Data and logic are encapsulated together.
Support	Supported by almost any OS.	Requires language support (e.g., Java synchronized).